

C# (.NET)

Introducción

C y C++ han sido lenguajes de programación muy utilizados porque proporcionan mucho control al programador, permitiéndole utilizar muchas funciones a bajo nivel y usar punteros. Sin embargo, aunque son lenguajes más potentes el tiempo de desarrollo es mayor frente a lenguajes más simples como Visual Basic. Microsoft se planteó crear un lenguaje para desarrollar aplicaciones rápidas, pero que también permitiese un gran control y que se integrase bien con el desarrollo de aplicaciones Web. El resultado fue C#, un lenguaje que combina las características de C, C++ y Java y utiliza la productividad del framework .NET

.NET Framework

En 1995 Microsoft comienza a dedicar esfuerzos al desarrollo en Internet, se dedica de lleno a integrar su plataforma Windows. Sin embargo tuvo que producir rápidamente herramientas y tecnologías para el desarrollo de aplicaciones en la red, un ejemplo de esto son las Active Server Pages (ASP). Aunque representa un concepto simple y fácilmente accesible para los nuevos desarrolladores, no soportan la orientación a objetos (y por tanto la herencia) y suponen una gran cantidad de secuencia de código. Además todas las herramientas de Microsoft adolecían de falta de integración y de facilidad de uso para el desarrollo Web.

Microsoft.NET es la primera plataforma de desarrollo diseñada desde sus cimientos teniendo en mente el desarrollo para Internet.

Muchos de los objetivos que Microsoft tuvo presente cuando diseñó .NET son un reflejo de las limitaciones identificadas en las herramientas y tecnologías anteriores:

- La tendencia en las aplicaciones comerciales iba hacia un modelo altamente distribuido, esto contrasta con el modelo dominante en ese momento en el que todos los elementos de una están localizados dentro de una única organización.
- Un objetivo básico fue lograr que el desarrollo para internet tenga un aspecto similar al desarrollo para otras plataformas. Un componente accedido en una red local o en Internet debería ser manipulado con una codificación muy similar a la que se utiliza para acceder al componente en un equipo local.

Es necesario que el desarrollo de interfaces de usuario sea lo más parecido posible para las dos alternativas: Internet o acceso local. Aunque las interfaces locales específicas de una plataforma siempre ofrecerán mayor flexibilidad que una interfaz basada en navegador, Microsoft.NET intenta que el desarrollo de ambos tipos sea lo más similar posible. El objetivo es que las interfaces de usuario basadas en Web sean más poderosas y flexibles, haciendo que se parezcan en aspecto y funcionalidad a las interfaces basadas en formularios utilizadas en aplicaciones locales.

- Los problemas de las DLL y los requerimientos de grandes procesos de instalación de las aplicaciones basadas en formularios, son sólo dos ejemplos de los problemas de la plataforma.

En 2014 Microsoft lanza **.NET Core** para convertirlo en multiplataforma y mejorar el rendimiento de las aplicaciones.

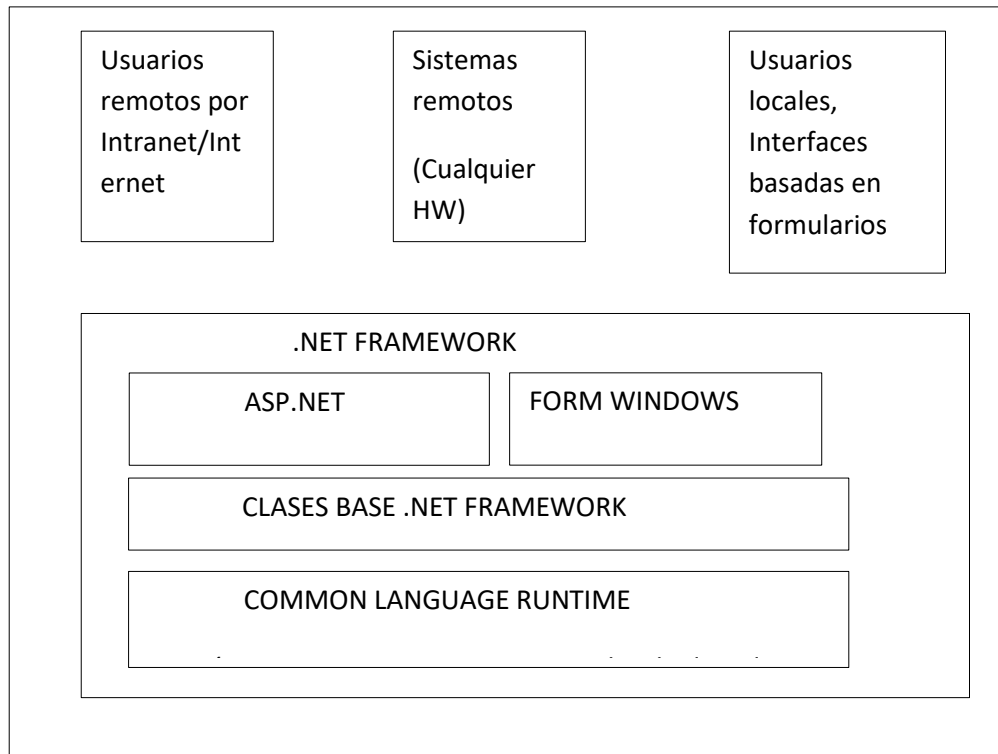
- A la versión vieja le llamamos **.NET Framework**, y esta llegó hasta la versión 4.8
- .NET Core es la versión nueva, y salió en 2016 en su versión 1
- .NET Core 3 => .NET 5
- Por tanto, a partir de .NET 5, tenemos .NET 6, .NET 7, .NET 8, y así, sucesivamente

.NET Framework es la versión vieja que sólo corre en windows

.NET Core es la nueva versión multiplataforma

La Estructura de Microsoft.NET

Veamos un diagrama de las principales capas de .NET que nos muestra que se coloca sobre un sistema operativo y suministra distintos modos de interfaz con el mundo exterior. Se debe observar que la arquitectura se ha creado para facilitar el desarrollo de aplicaciones Internet, de manera que esta tarea sea similar al desarrollo de una aplicación clásica de escritorio:



.NET Framework envuelve al sistema operativo, aislando al software desarrollado con .NET de las tareas específicas del s. o., por ejemplo, la gestión de archivos y la reserva de memoria. Es un entorno para construir, instalar y ejecutar aplicaciones y servicios web. Consta de dos elementos principales: CLR y la librería de clases.

Common Language Runtime

El entorno de ejecución común del lenguaje (CLR) es el corazón de la plataforma. Es el motor de ejecución que carga, ejecuta y gestiona el código intermedio (MSIL) que ha sido compilado. Esto significa que hay dos niveles de compiladores, el compilador del lenguaje toma el código fuente y crea el código MSIL. Este código es transportable a

cualquier otra plataforma .NET y en tiempo de ejecución se compila a código binario correspondiente a la máquina en donde se ejecuta el código.

Biblioteca de clases. NET Framework

La capa siguiente suministra los servicios y modelos de objetos para los datos, la entrada/salida,....

Esta biblioteca de clases contiene miles de clases e interfaces, todos los lenguajes .NET disponen de estas clases Framework.

Ejemplo

“Calculo de raiz cuadrada en Visual y C#

```
Dim dblNumber As Double =200
Dim dblSquareRoot As Double
DblSquareRoot=System.Math.Sqrt(dblNumber)
```

```
Double dblNumber =200;
Double dblSquareRoot ;
DblSquareRoot=System.Math.Sqrt(dblNumber);
```

Interfaces de usuario y programas

En cierto sentido, la capa superior de la plataforma es una extensión de las clases bases del Framework. Está compuesta por las nuevas interfaces de usuarios de programas:

Windows Forms: motor de formularios independiente del lenguaje que lleva a todos los lenguajes .NET la capacidad de arrastrar y soltar, propia del diseño de Visual Basic, y permite desarrollar interfaces basadas en formularios con escaso o ningún acceso a la API de Win32.

Web Forms: lleva a las interfaces basadas en la Web un modelo de programación similar al desarrollo basado en formularios de VB6. Las interfaces de usuarios creadas con Web Forms también incorporan independencia del navegador. Estos formularios Web representan un gran paso hacia delante

ASP.NET: desarrollo backend

Los Servicios Web: nos permiten invocar y utilizar componentes remotos, posiblemente en ejecución en sistemas operativos diferentes, podemos así construir aplicaciones altamente distribuidas. Estos servicios dependen totalmente de XML para realizar las interfaces con objetos remotos (también ADO.NET depende de XML de manera importante).

Namespaces

Otro concepto importante de la plataforma son los espacios de nombres o namespaces. Un namespace es el medio proporcionado por la plataforma para organizar las clases dentro del entorno, agrupándolas de un modo lógico y jerárquico. Para acceder desde el código de una aplicación, a una clase contenida dentro de un espacio de nombre, debemos importarlo (using en C#).

Para hacer referencia a una clase de en un espacio de nombre, se pone el espacio de nombre y la clase separados por un punto. En el caso de acceder a una clase que se encuentra con varios espacios de nombre de profundidad, especificaremos dichos espacios de nombres separados por un punto, y al final la clase.

En el caso de instanciar un objeto directamente en el código, escribiremos los espacios de nombre y al final el nombre de la clase.

Si importamos los espacios de nombre, no debemos poner el nombre de la clase, sino que debemos terminar con el espacio de nombres que contiene la clase que necesitamos.